SVKM's NMIMS

Mukesh Patel School of Technology Management and Engineering

Department of Computer Engineering

A REPORT ON

"Design and Development of a Distributed Real-Time Market Data Streaming Backend at Finseal Software Pvt. Ltd."

By

Riyansh Sachdev

70012200032

B076

BTech Computer Engineering

Finseal Software Pvt. Ltd.

05/01/2026 – 04/07/2026

A TECHNICAL INTERNSHIP REPORT ON

"Design and Development of a Distributed Real-Time Market Data Streaming Backend at Finseal Software Pvt. Ltd."

By:

Riyansh Sachdev

70012200032

A Report submitted in partial fulfillment of requirements of 4-year BTech (Computer Engineering) Program of Mukesh Patel School of Technology, Management and Engineering, NMIMS

Under the supervision of:

Industry Mentor

Mr. Darshan Doshi

(Chief Executive Officer, Finseal Software Pvt. Ltd.)

Faculty Mentor

Dr. Soni Sweta

(Computer Engineering Department, MPSTME)

COMPLETION CERTIFICATE ISSUED BY COMPANY

Finseal

Finseal Software Pvt Ltd

To Whom It May concern,

This is certify that **Mr. Riyansh Sachdev (B076)** is completing his training & project as a part of Technical Internship in our company as mentioned below.

i. **Project Title:** Design and Development of a Distributed Real-Time Market Data Streaming Application
ii. **Date of Joining: 5-01-2026**
iii. **Date of Completion:4-07-2026**

In partial fulfillment of VIII Semester Technical Internship for B. Tech program of Mukesh Patel School of Technology Management & Engineering, Narsee Monjee Institute of Management Studies (NMIMS) (Deemed-to-be University), Mumbai.

Thanks,

Finseal Software Pvt Ltd

FINSEAL SOFTWARE PRIVATE LIMITED Mumbai

Authorized Signatory

Mr. Darshan Doshi
Industry Mentor
Date:02-05-2026
Finseal Software Services Pvt. Ltd.

916704039
accounts@f seal.in
1301, B-wi , O2 business Centre, Mulund west, Mumbai-80

CERTIFICATE OF COMPLETION

SVKM'S NMIMS

Mukesh Patel School of Technology Management & Engineering,

Vile Parle (W), Mumbai – 400056.

TECHNICAL INTERNSHIP REPORT

Semester VIII – BTech

Submitted in Partial Fulfilment of requirements for Technical Project/Training for VIII Semester BTech

Name of the Student: RIYANSH SACHDEV

Roll No. B076 & Batch: Batch of 2026

Academic year: 2025-2026

Name of the Discipline: BTech (Computer Engineering)

Name of the Company: Finseal Software Pvt. Ltd.

Address of company: 1301/1302, B-Wing, O2 Business Centre, Asha Nagar, Mulund (W), Mumbai – 400080

Training Period: 05/01/2026 to 04/07/2026

THIS IS TO CERTIFY THAT

Student Name: MR. RIYANSH SACHDEV

has satisfactorily completed his Training/Project Work, submitted the training report, and appeared for the Presentation & Viva as required.

External Examiner Internal Examiner Head of Dept. Chairperson/Dean

Date:

Place:

Seal of the University:

TABLE OF CONTENTS

ABSTRACT

This final report presents a comprehensive technical overview of the end-to-end Software Development Life Cycle (SDLC) executed during a six-month professional internship at Finseal Software Pvt. Ltd. The tenure was characterised by the design, implementation, load-testing and CI/CD-driven deployment of TrradeFlow — a distributed, low-latency market data backend written in Go that ingests live Binance WebSocket feeds across 300 USDT trading pairs, executes six concurrent streaming analytics engines, and pushes indicator updates to authenticated WebSocket consumers in real time.

The architectural cornerstone of the project is a strict two-binary split — a dataserver that owns all computation (ingestion, indicator math, persistence) and a clientserver that owns all client-facing I/O (WebSocket fan-out, REST history, authentication). The two binaries share no internals; the primary coupling is a NATS JetStream event bus (stream ANALYTICS, subject analytics.events) over which the dataserver publishes fixed-width binary frames (22–46 bytes) and a durable JetStream consumer on each clientserver fans them out through 16 parallel worker goroutines. A secondary Redis Pub/Sub channel (chat:events) is used between clientserver replicas for cross-instance broadcast. Six analytics engines — SMA, EMA, RSI (Wilder's), MACD, Bollinger Bands and OHLC — are registered through a pluggable indicator registry and run at dual resolutions (1-second tick-level and 1-minute aggregates).

A primary engineering focus was high-availability data infrastructure. Two independent Redis primary–replica pairs were deployed, each protected by a Redis Sentinel for automatic failover, with a custom-built load balancer routing writes to the least-loaded healthy primary and a scatter–gather read layer fanning queries across replicas to take the fastest response. Session/auth storage is deliberately pinned to a single primary to guarantee read-your-own-writes consistency for authentication tokens. The dataserver itself uses a Redis SETNX-lease leader-election mechanism (10 s TTL, 3 s renewal) so that multiple replicas can run in standby with only one active leader executing the engines at a time. A three-tier history layer (1-minute warm cache → 1-hour rollup → Postgres cold storage) provides instant history bursts on room join, with a non-blocking background warmer pre-loading Redis from Postgres on user login.

Authentication is token-based with UUID session tokens stored in Redis (24 h TTL), wrapped by an AuthMiddleware that validates the Bearer token and injects clientID into the request context; WebSocket upgrades are rejected pre-handshake (HTTP 4001) for invalid tokens. The system was load-tested with Locust to 500 concurrent clients across 300 instruments delivering 776,000 messages with zero drops at ~100 ms average latency and a stable ~10 MB heap. The project was deployed through a GitHub Actions CI/CD pipeline (build, vet, golangci-lint, Dockerised image push to GHCR, SSH-driven docker compose pull && up) onto a containerised production

environment described by a single docker-compose.yml that brings up all six Redis nodes, NATS JetStream, Postgres, both Go binaries and an nginx least_conn reverse proxy with a health-check-gated startup order.

Chapter 1: INTRODUCTION

1.1 About the Company

Finseal Software Services Pvt. Ltd. is a Mumbai-based technology company operating at the intersection of financial technology and backend systems engineering. The organisation specialises in designing and delivering software platforms that support real-time market data processing, trading-related workflows and scalable backend infrastructure for brokerage and capital-market clients.

Finseal emphasises clean system architecture, deterministic performance and operational reliability, with a particular focus on applications that demand low-latency communication, event-driven processing and high concurrency. The company follows modern software engineering practices — version-controlled workflows, code review, CI/CD-based delivery and infrastructure-as-code — and provides an environment oriented towards practical learning in distributed systems, real-time application design and production-grade Go development. Its core expertise spans backend service architecture, in-memory and persistent data stores, and white-label trading-platform delivery for downstream brokers.

1.2 Overview

The primary objective of this internship was to design, build and operationalise TrradeFlow — a production-grade distributed backend capable of ingesting live exchange data, computing real-time technical indicators and broadcasting the results to authenticated WebSocket consumers at scale. The initiative centred on moving from a conventional monolithic ingestion-and-broadcast design to a decoupled, event-driven architecture with strong separation between computation and client I/O, coupled by a durable NATS JetStream message bus.

The system ingests the Binance WebSocket feed for 300 USDT trading pairs, fans the stream into six streaming analytics engines and persists results to both Redis (for hot reads) and Postgres (for cold storage). To address the operational risk inherent in real-time financial pipelines, Redis Sentinel was deployed in a two-pair topology with a custom load balancer and a scatter-gather read layer, validated live for zero consumer disconnections during a primary-node failure. A Redis-lease-based leader election mechanism in internal/leader allows multiple dataserver replicas to run in standby with only one elected leader executing the engines, enabling fast failover without double-publishing. The internship scope expanded over the tenure from pure backend engineering into full-stack ownership — encompassing a vanilla-JS dashboard rendered with LightweightCharts, an nginx reverse-proxy layer for WebSocket-safe load balancing, a Locust-based load-testing harness, and a complete GitHub Actions CI/CD pipeline that ships Docker images to GHCR and deploys them via SSH to a remote host.

Chapter 2: PROJECT DETAILS

2.1 Project Aim

The TrradeFlow project aimed to:

- **Real-Time Market Data Pipeline:** Build a low-latency Go backend capable of ingesting and routing live Binance price feeds for 300 instruments without drops.
- **Streaming Analytics:** Deliver six concurrent indicator engines (SMA, EMA, RSI, MACD, Bollinger Bands, OHLC) at dual time resolutions (1s+1m), driven by a pluggable indicator registry.
- **Architectural Decoupling:** Enforce a strict two-binary split (dataserver for compute, clientserver for client I/O) coupled by a NATS JetStream event bus carrying compact binary analytics frames, to enable independent scaling and fault isolation.
- **High-Availability Storage:** Engineer a resilient Redis layer with two primary–replica pairs, Sentinel-based automatic failover, a write-side load balancer and a scatter-gather read layer; plus dataserver leader election via a Redis SETNX lease for active/standby operation.
- **Authenticated Real-Time Delivery:** Implement token-based authentication for both REST endpoints and WebSocket upgrades, with read-your-own-writes session consistency.
- **Operational Readiness:** Validate the system under realistic load (500 concurrent clients, 776 K messages, zero drops) and ship it through an automated CI/CD pipeline into a containerised production environment.

2.2 Project Scope

- **Backend Development:** Implementing the dataserver and clientserver binaries in Go 1.25.5, using gorilla/websocket for real-time transport, pgx/v5 (pooled via pgxpool) for Postgres access and nats.go (JetStream) for the analytics event bus.
- **Analytics Engines:** Designing the indicator registry and implementing SMA, EMA, RSI (Wilder's smoothing), MACD (12/26/9), Bollinger Bands (window = 20, k = 2) and OHLC engines with consistent 1 s / 1 m resolutions.
- **Redis Infrastructure:** Deploying two primary–replica Redis pairs with Sentinel failover, building a custom load balancer for write routing, and implementing a scatter-gather read path with replica fan-out.
- **Event Bus:** Operating NATS JetStream as the primary inter-binary event bus — stream ANALYTICS (memory storage, 5-minute MaxAge, 64 KB MaxMsgSize), durable consumer on each clientserver with MaxAckPending=4096 and 16 fan-out worker goroutines; ack-before-process semantics chosen deliberately because stale market ticks are worse than dropped ones.
- **Binary Frame Protocol:** Designing a fixed-width binary wire format in internal/binary (22 B scalar, 38 B BB/MACD, 46 B OHLC) with zero-allocation decode — roughly 3× smaller than the prior JSON envelopes and decoded into JSON only at the WebSocket boundary.
- **Leader Election:** Implementing Redis SETNX-based leader election in internal/leader with a 10 s lease and 3 s renew interval, plus Lua-guarded renewal/release so a deposed leader cannot accidentally clobber the new owner's key.
- **History Layer:** Engineering the three-tier history store (1 m warm cache 48 h TTL → 1 h rollup 7 d TTL → Postgres cold storage) and the on-login background warmer.
- **Authentication & Session Management:** Building the AuthMiddleware, UUID-token issuance, Redis-pinned session storage and pre-upgrade WebSocket token validation (HTTP 4001 close code).
- **WebSocket Protocol:** Designing the dual push/pull protocol — room-join push for all six indicators of an instrument, and topic-level subscribe/unsubscribe — on a single connection.
- **Frontend Dashboard:** A single-file vanilla-JS dashboard (frontend/index.html) using LightweightCharts v4.1.3 for candlestick + Bollinger overlay + RSI/MACD combo chart, with per-instrument caching and auto-reconnect logic.

- **Reverse Proxy & Routing:** Configuring nginx with least_conn for long-lived WebSocket connections across two clientserver replicas, and round-robin for stateless /metrics endpoints.
- **Load Testing:** Authoring a Locust-based harness (locustfile.py) and a Go-based WS load tester (ws-load-tester) to validate throughput, latency and heap stability under 500 concurrent clients.
- **CI/CD:** Splitting the pipeline into a ci.yml workflow (Go 1.25.5 build + vet + golangci-lint v1.64.8 + Docker image validation) and a deploy.yml workflow (GHCR push tagged with SHA + latest + remote docker compose pull && up).
- **Containerised Stack:** A docker-compose.yml that brings up six Redis nodes (two pairs + two sentinels), NATS JetStream (-js), Postgres 16, both Go binaries and nginx, with a health-check chain that guarantees startup order: Redis → sentinels → NATS → Postgres → dataserver → clientserver → nginx.

2.3 Project Methodology

Development followed an iterative, milestone-driven methodology aligned with Agile principles, with weekly checkpoints with the industry mentor:

- **Requirement Engineering:** Translating the business need ("real-time cryptocurrency price + indicator stream for 300 instruments, authenticated, multi-consumer") into a concrete event-flow specification covering ingestion, computation, fan-out and persistence boundaries.
- **Architecture & Design:** Producing the two-binary split design, the Redis topology diagram (two pairs + Sentinels + LB), the three-tier history strategy and the WebSocket dual-mode protocol.
- **Incremental Implementation:** Building the ingestion adapter first, then the indicator registry and individual engines, followed by the Redis storage layer, the Pub/Sub bridge, the client-server WebSocket hub and finally the authentication middleware.
- **Resilience Engineering:** Bringing up Sentinel, the custom write-side load balancer and the scatter-gather read path, then deliberately killing primaries under load to validate failover with zero consumer drops.
- **Performance & Load Testing:** Running Locust scenarios at 200 → 500 concurrent clients across 300 instruments, measuring drop rate, latency distribution and heap stability via the /metrics endpoint.
- **Containerisation & Deployment:** Authoring Dockerfile.dataserver, Dockerfile.clientserver and docker-compose.yml; setting up nginx for WebSocket-safe least_conn routing; pushing images to GHCR; and SSH-deploying via GitHub Actions.
- **Documentation & Handoff:** Producing the architectural README, endpoint reference, deployment runbook, CI/CD secret inventory and Go-version compatibility notes.

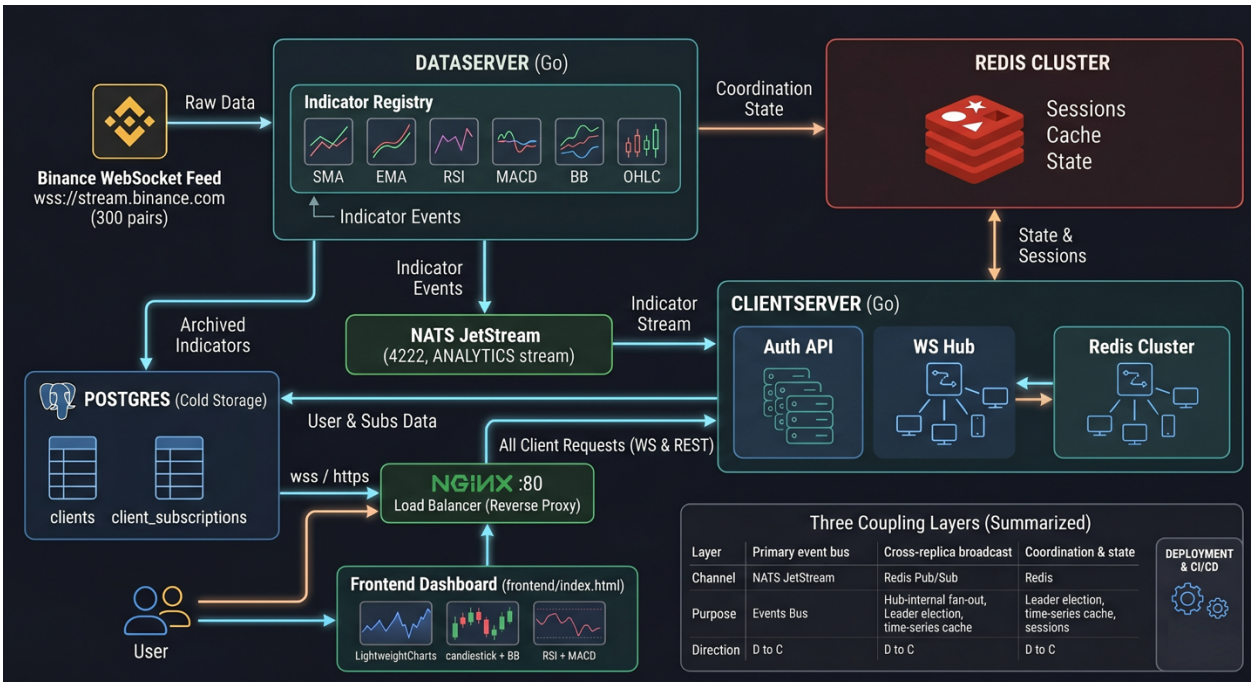
2.4 Project Timeline

Week	Period	Major Tasks & Accomplishments
1	Mon 05/01/26 – Fri 09/01/26	Requirement gathering with industry mentor; architecture design (two-binary split, event-bus choice); Go module bootstrap and repository layout.
2	Mon 12/01/26 – Fri 16/01/26	Binance WebSocket ingestion adapter (feed-adapter); exchange normalisation layer for 300 USDT instruments.
3	Mon 19/01/26 – Fri 23/01/26	Basic price routing into the indicator registry; project skeletons for the engine workers.
4	Mon 26/01/26 – Fri 30/01/26	Indicator registry abstraction; SMA engine (window = 20); dual-resolution (1s + 1m) compute pipeline.
5	Mon 02/02/26 – Fri 06/02/26	EMA engine (window = 20); Redis sorted-set persistence; engine-output plumbing into the publisher.
6	Mon 09/02/26 – Fri 13/02/26	RSI engine with Wilder's smoothing (period = 14); per-engine input-channel back-pressure handling.
7	Mon 16/02/26 – Fri 20/02/26	MACD engine (12/26/9); multi-value indicator payload design (macd_line, signal_line, histogram).
8	Mon 23/02/26 – Fri 27/02/26	Bollinger Bands engine (window = 20, k = 2.0); OHLC aggregator with proper open/high/low/close merge.
9	Mon 02/03/26 – Fri 06/03/26	Per-indicator Postgres schema design (sma, ema, rsi, ohlc, bb, macd) and pgx/v5 persistence layer via pooled pgxpool.
10	Mon 09/03/26 – Fri 13/03/26	Three-tier history store (1 m warm cache 48 h TTL → 1 h rollup 7 d TTL → Postgres cold storage); hourly rollup job.
11	Mon 16/03/26 – Fri 20/03/26	Two-binary split: extraction of dataserver from clientserver; clean module boundaries; shared interfaces only.
12	Mon 23/03/26 – Fri 27/03/26	NATS JetStream bus on subject analytics.events; fixed-width binary wire format (22–46 B frames) in internal/binary; durable JetStream consumer with 16 fan-out workers per clientserver.
13	Mon 30/03/26 – Fri 03/04/26	WebSocket dual push/pull protocol via gorilla/websocket; cross-replica Redis Pub/Sub (chat:events) for multi-clientserver broadcast.
14	Mon 06/04/26 – Fri 10/04/26	Authentication system: UUID session tokens in Redis (24 h TTL), AuthMiddleware, Bearer-token REST validation, pre-upgrade WebSocket token check with HTTP 4001 rejection; /register /login /logout /subscribe /unsubscribe /subscriptions endpoints.
15	Mon 13/04/26 – Fri 17/04/26	High-availability Redis: two primary–replica pairs, Sentinel × 2, custom write-side load balancer, scatter-gather read layer; session storage pinned to pair2Primary for read-your-own-writes.

16	Mon 20/04/26 – Fri 24/04/26	Dataserver leader election via Redis SETNX lease (10 s TTL, 3 s renew, Lua-guarded renew/release); failover drills under load; frontend dashboard (frontend/index.html) with LightweightCharts candlestick + BB overlay + RSI/MACD combo chart.
17	Mon 27/04/26 – Fri 01/05/26	Load testing with Locust and ws-load-tester (500 clients, 776 K messages, zero drops); nginx least_conn reverse proxy; Ristretto L1 cache; Dockerisation (two Dockerfiles + docker-compose.yml); GitHub Actions CI/CD split into ci.yml and deploy.yml; GHCR publishing; SSH-based remote deploy; documentation handoff.

2.5 Technologies Used

- **Language & Runtime:** Go 1.25.5 (chosen for native concurrency, low GC pressure and strong WebSocket library support).
- **Real-Time Transport:** gorilla/websocket for client delivery (/ws); the dataserver consumes the Binance market WebSocket stream directly via the feed-adapter.
- **Event Bus:** NATS JetStream (nats.go v1.38.0) as the primary inter-binary event bus — stream ANALYTICS, subject analytics.events, memory storage, durable per-consumer cursors.
- **In-Memory Store & Coordination:** Redis 7 (Sorted Sets for time-series indicators; Pub/Sub channel chat:events for cross-instance clientserver broadcast; Sentinel for failover; SETNX lease for dataserver leader election); two primary–replica pairs (mymaster, mymaster2) plus a custom write-side load balancer.
- **Wire Format:** Fixed-width binary frames (22–46 B) defined in internal/binary — zero-allocation decode path; replaces an earlier JSON envelope format that was 3–5× larger.
- **L1 Cache:** Ristretto for in-process hot-path caching of frequently-requested history slices.
- **Persistent Store:** PostgreSQL 16 accessed via pgx/v5 and a pooled pgxpool (max 50, min 4 connections); per-indicator tables (sma, ema, rsi, ohlc, bb, macd) plus clients and client_subscriptions; schema bootstrapped by init.sql.
- **Reverse Proxy:** nginx with least_conn upstream balancing for /ws and the REST surface, round-robin for /metrics; CORS handled in Go to avoid double-header conflicts.
- **Frontend:** Vanilla JavaScript + LightweightCharts v4.1.3, served as a single-file dashboard with token-in-memory auth and auto-reconnect.
- **Load Testing:** Locust (locustfile.py) and a Go-based custom harness (ws-load-tester, ws-pull-tester).
- **Containerisation:** Docker (Dockerfile.dataserver, Dockerfile.clientserver) + Docker Compose.
- **CI/CD:** GitHub Actions — ci.yml (build, go vet, golangci-lint v1.64.8, Docker image build) on every push; deploy.yml (GHCR push tagged with SHA + latest, SSH docker compose pull && up -d) on main.
- **Image Registry & Deployment:** GitHub Container Registry (GHCR); remote host (Oracle Cloud Free Tier / Hetzner CAX11) with Docker Compose.



(Figure 1: Architectural flow of the project including the tech stack.)

2.6 Future Scope

The modular, event-driven architecture established during the internship admits several high-value extensions:

- **TimescaleDB Migration:** Replacing the flat per-indicator Postgres tables with TimescaleDB hypertables would yield order-of-magnitude improvements in time-bucketed aggregation queries and enable native continuous aggregates for the hourly rollup job.
- **Horizontal Hub Sharding:** The current clientserver hub holds all rooms in a single process; partitioning rooms across multiple hubs (with a consistent-hash router) would let the system scale beyond the single-node WebSocket connection ceiling.
- **Additional Indicators:** Adding ATR, Stochastic Oscillator and VWAP engines through the existing indicator registry — no changes required to the worker, routing or persistence layers.
- **Event Replay & Durability:** Switching the existing JetStream stream from memory storage to file storage (and lengthening MaxAge) would convert the bus from real-time-only into a replayable history that surviving restarts and rolling deploys without losing the most recent five minutes of analytics events.
- **Authentication Hardening:** Moving from UUID session tokens to short-lived JWTs with refresh-token rotation, plus per-IP rate-limiting on /login and /ws.
- **Observability Stack:** Wiring the existing /metrics endpoint into Prometheus + Grafana for engine backlogs, drop rates, heap and goroutine counts, with alerts on indicator-engine staleness.
- **Multi-Exchange Ingestion:** Generalising the Binance adapter into a pluggable exchange-adapter interface to ingest Coinbase, Kraken and OKX feeds in parallel and emit normalised events to the same Pub/Sub bus.

Chapter 3: CONCLUSION

3.1 Learnings

This internship was a substantial learning experience in distributed systems engineering and real-time backend development, translating textbook concepts (pub/sub, scatter-gather, failover, fan-out) into a working, load-tested production system. Key technical and professional takeaways include:

- **Distributed Architecture & Event-Driven Design:** Designing the strict dataserver/clientserver split coupled by a NATS JetStream event bus (with Redis Pub/Sub as a secondary cross-replica broadcast channel) cemented my understanding of decoupled producer–consumer systems, durable messaging, fault isolation and the operational benefit of binaries that can be scaled, restarted and redeployed independently.
- **Wire-Format Engineering:** Designing the fixed-width binary frame protocol (22–46 B, zero-allocation decode) replaced an earlier JSON-envelope format and taught me how to balance human-readable wire formats against measurable wins in bandwidth and GC pressure under sustained 20 K msg/s load.
- **Coordination & Leader Election:** Implementing Redis-lease-based leader election with a Lua-guarded renew/release path was a sharp lesson in distributed consensus primitives — in particular, why naive EXPIRE/DEL without owner verification is unsafe under clock skew or network partitions.
- **High-Availability Engineering:** Building the two-pair Redis topology with Sentinel failover, a custom write-side load balancer and a scatter-gather read layer taught me how to reason about availability versus consistency trade-offs — including the deliberate choice to pin session storage to a single primary for read-your-own-writes correctness.
- **Concurrency in Go:** Implementing six concurrent indicator engines, a fan-out hub for hundreds of WebSocket clients and a non-blocking background warmer deepened my fluency with goroutines, channels, context propagation and the sync primitives needed to keep state safe under load.
- **Real-Time Protocol Design:** Designing the dual push/pull WebSocket protocol (room-join push for all six indicators plus topic-level subscribe/unsubscribe on the same connection) taught me how to balance protocol simplicity against client flexibility.
- **Authentication for Stateful Connections:** Wiring token validation into the WebSocket upgrade path (rejecting with HTTP 4001 before the upgrade completes) and the corresponding client-side reconnect logic gave me a practical model for authenticated long-lived connections.

- **Performance & Load Testing:** Using Locust and a custom Go harness to drive 500 concurrent clients across 300 instruments at 776 K messages with zero drops and a stable ~10 MB heap, then reading the /metrics endpoint to diagnose backlogs, made performance work concrete rather than theoretical.
- **Production Operations:** Owning the full deployment chain — Dockerfiles, Docker Compose, nginx least_conn for WebSockets, GitHub Actions CI/CD with GHCR and SSH-based deploys — bridged the gap between "code works on my Mac" and "code runs reliably in a remote container".
- **Build-System Discipline:** Diagnosing the Go 1.25.5 transitive-dependency requirement (golang.org/x/sync, x/text, x/mod) and aligning CI, Dockerfiles and lint tooling around it was a sharp lesson in keeping the toolchain coherent across local, CI and production environments.

3.2 Conclusion

The internship at Finseal Software Pvt. Ltd. concluded with the successful delivery of TrradeFlow — a production-ready, distributed real-time market data backend that demonstrably handles 500 concurrent authenticated clients across 300 instruments at sub-100-millisecond average latency with zero message drops and a stable ~10 MB heap. The system is split into two cleanly-decoupled Go binaries communicating via a NATS JetStream event bus carrying compact binary frames (with a secondary Redis Pub/Sub channel for cross-replica clientserver broadcast), backed by a Sentinel-protected two-pair Redis topology with custom load balancing, scatter-gather reads and SETNX-lease leader election; persisted to PostgreSQL through a three-tier history store, fronted by an nginx least_conn reverse proxy and delivered through a fully automated GitHub Actions CI/CD pipeline that ships Docker images to GHCR and deploys them to a remote host via SSH-driven docker compose pull && up.

Beyond the immediate deliverable, the project was a complete end-to-end exercise in modern backend engineering — architecture, concurrency, persistence, real-time transport, authentication, observability, containerisation and deployment — under the realistic constraints of a fintech-domain workload. Transforming an abstract requirement ("stream live market data and indicators to authenticated consumers") into a load-tested, containerised, continuously-deployed system has been an invaluable milestone in my journey as a Computer Engineer, and the architectural foundation laid during the internship is well-positioned to absorb the future enhancements outlined above.

Chapter 4: REFERENCES

[1] Synadia / NATS.io, "NATS JetStream — Persistent Messaging, Streams and Durable Consumers," [Online]. Available: https://docs.nats.io/nats-concepts/jetstream.

[2] Binance, "Binance WebSocket Market Streams API Reference," [Online]. Available: https://binance-docs.github.io/apidocs/spot/en/#websocket-market-streams.

[3] Redis Ltd., "Redis Sentinel Documentation — High Availability and Automatic Failover," [Online]. Available: https://redis.io/docs/management/sentinel/.

[4] Redis Ltd., "Redis Pub/Sub and Sorted Set Reference," [Online]. Available: https://redis.io/docs/interact/pubsub/.

[5] Gorilla Web Toolkit, "gorilla/websocket — A Fast, Well-Tested WebSocket Implementation for Go," [Online]. Available: https://github.com/gorilla/websocket.

[6] Jack Christensen et al., "pgx — PostgreSQL Driver and Toolkit for Go (v5)," [Online]. Available: https://github.com/jackc/pgx.

[7] Dgraph Labs, "Ristretto — A High-Performance Memory-Bound Go Cache," [Online]. Available: https://github.com/dgraph-io/ristretto.

[8] TradingView, "Lightweight Charts™ v4.1.3 API Reference," [Online]. Available: https://tradingview.github.io/lightweight-charts/.

[9] F5/NGINX, "Using NGINX as a WebSocket Proxy — least_conn Load Balancing," [Online]. Available: https://www.nginx.com/blog/websocket-nginx/.

[10] Locust.io, "Locust — An Open Source Load Testing Tool," [Online]. Available: https://locust.io/.

[11] GitHub Inc., "GitHub Actions and GitHub Container Registry (GHCR) Documentation," [Online]. Available: https://docs.github.com/en/actions.

Mr. Darshan Doshi

Chief Executive Officer, Finseal Software Pvt. Ltd.

02-05-2026